

TD n° 1

Le format Portable Gray Map (PGM)

Eric REMY
eric.remy@univ-amu.fr

Version du 13 septembre 2023

Exercice

Pour ce TD, on suppose que vous avez déjà implémenté en C++ les parties de la classe `GrayImage` vues en cours. On va s'intéresser à des données au format « Portable Gray Map » (PGM). Un fichier de ce type répond à la structure suivante :

- un en-tête conventionnel et obligatoire, nommé habituellement un *magic number*, formé de la chaîne de caractères « P5 » suivi d'un caractère ASCII « Line Feed » ;
- zéro, une ou plusieurs lignes de commentaires commençant obligatoirement chacune par le caractère ASCII « # » et allant jusqu'à un caractère « Line Feed » final ;
- une ligne de texte ASCII comportant l'écriture en base 10 de la largeur puis de la hauteur de l'image séparées par un espace, le tout suivi d'un « Line Feed » ;
- zéro, une ou plusieurs lignes de commentaires supplémentaires sur le même principe que précédemment ;
- une ligne de texte comportant l'écriture en base 10 de la valeur maximale qu'un pixel peut prendre (pour nos TP, ce sera 255) suivie d'un caractère « Line Feed » ;
- une suite d'octets binaires représentant les valeurs de chaque pixel en commençant par celui en haut et à gauche de l'image, suivi par celui immédiatement à sa droite, etc. Les lignes sont envoyées à la suite les unes des autres, de haut en bas dans l'image.

La figure 1 montre une image de 5 colonnes par 4 lignes de pixels, avec en surimpression ses valeurs d'intensité lumineuses (en décimal et en hexadécimal) ainsi que son affichage dans un visualiseur hexadécimal/ASCII. À titre de référence, une table ASCII est donnée à la figure 2.

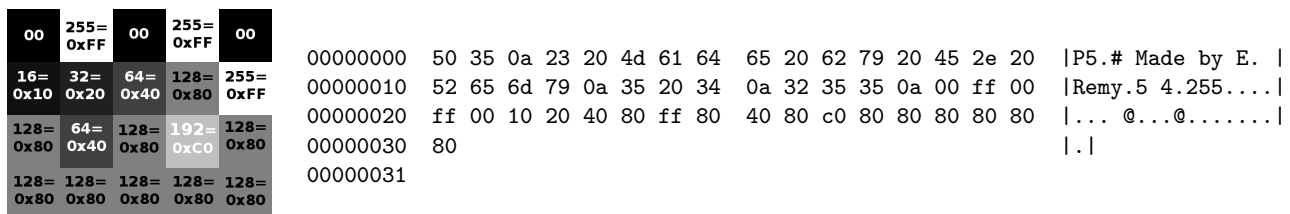


FIGURE 1 – Un exemple d'image PGM simple ainsi que sa structure en hexadécimal et ASCII.

1. Vérifiez que vous identifiez bien chaque champ de l'en-tête et que vous reconnaissez les valeurs des pixels dans le fichier.
2. Implantez la méthode d'instance « `void GrayImage::writePGM(std::ostream&) const` » qui écrit l'image courante en suivant les spécifications du format PGM sur le flux binaire reçu en paramètre. Vous indiquerez « Image sauvegardée par XXX pour les TP de RepCod. » (où XXX est votre nom) comme première chaîne de commentaire. Vous êtes libres d'ajouter une ou plusieurs lignes de commentaires supplémentaires.
3. Implantez une fonction « `void skip_line(std::istream&)` » qui lit des caractères depuis le flux reçu en paramètre jusqu'à lire le caractère « Line Feed ».
4. Implantez une fonction « `void skip_comments(std::istream&)` » qui lit d'éventuels lignes de commentaires. Pour cela, elle lit un caractère et si ce caractère n'est pas « # », c'est-à-dire s'il n'y a pas de commentaire, elle le replace sur le flux d'entrée grâce à l'opération « `istream& std::istream::putback(char c)` » puis elle se termine (comme si elle n'avait jamais lu ce caractère). Par contre, si le caractère est « # », elle saute ensuite le reste de la ligne jusqu'au « Line Feed » qui marque sa fin. Cette opération de saut d'une ligne de commentaire doit être répétée tant qu'on trouve bien un « # » au début de chaque ligne.
5. Implantez la fonction de classe « `static GrayImage* GrayImage::readPGM(std::istream& is)` » qui fabrique une nouvelle image en niveaux de gris à partir des données contenues dans le flux de fichier binaire passé en paramètre.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

FIGURE 2 – Table des codes ASCII standards.

Rappel

Toute fonction membre dispose automatiquement des droits d'accès aux membres privés de sa classe. En C++, les données et les fonctions membres « classiques » (sans précision) sont des membres d'*instances*, c'est-à-dire qu'elles sont attachées à chaque instance de leur classe. Si nous plaçons notre fonction « readPGM() » comme membre d'instance, nous allons nous retrouver dans la situation paradoxale consistant à avoir besoin de disposer déjà d'une instance de GrayImage pour pouvoir charger un fichier et créer une nouvelle instance : c'est le serpent qui se mord la queue !

On va donc préférer attacher notre fonction à la classe elle-même, c'est-à-dire en faire une fonction membre de classe, en utilisant le mot clé « static » dans la déclaration de classe. Ainsi, il devient possible d'appeler la fonction sur la classe elle-même, sans avoir d'instance préalable, tout en ayant les droits d'accès aux membres privés. Révisez votre C++ si nécessaire.